

Principy objektového stylu

Karel Richta a kol.

Katedra technických studií
Vysoká škola polytechnická Jihlava

© Karel Richta, 2023

Objektově-orientované programování, OOP
03/2023, Lekce 2

<https://moodle.vspj.cz/course/view.php?id=201479>



Vysoká škola
polytechnická
Jihlava

Různé styly řešení problémů

- Zápis programu realizuje určitou funkci či algoritmus, které představují řešení určitého problému.
- K řešení problému můžeme dospět různými způsoby – styly.
- Dva základní styly, kterými se zabýváme jsou strukturovaný styl a objektově-orientovaný styl.
- Každý z nich se hodí na řešení trochu jiných problémů.
- Strukturovaný styl se hodí pro řešení procesních problémů, kdy řešení skládáme z posloupností jednotlivých kroků. Jednodušeji se takové řešení vyjadřuje v jazyce C.
- Objektově-orientovaný styl se hodí spíše pro řešení různých informačních, či návrhových systémů. Jednodušeji se takové řešení vyjadřuje v jazyce C++.

Stručný úvod do strukturovaného stylu

- Strukturovaný model výpočtu si představuje řešení jako sadu strukturovaných dat, nad kterými pracují strukturované funkce.
- Strukturovaný zde znamená složený z jednodušších složek systematickým způsobem.
- Na úrovni dat to znamená, že data se skládají z primitivních prvků, nebo dat strukturovaných z jiných prvků do polí nebo struktur.
- Na úrovni funkcí to znamená, že funkce se skládají z primitivních příkazů, nebo příkazů strukturovaných pomocí do sekvencí (posloupností), selekcí (podmíněných konstrukcí) nebo iterací (cyklů).
- Příklad: Představme si řešení fronty zákazníků. Především navrhujeme datovou reprezentaci fronty. Nad ní pak definujeme mimo jiné funkci `zarad_zakaznika`, která do fronty zařadí dalšího zákazníka. Systém, který simuluje obsluhu zákazníků vytvoří nového zákazníka (data typu `ZAKAZNIK`) a nechá ho zařadit do fronty zákazníků. Z fronty pak vybírá zákazníky, které bude obsluhovat (pomocí dalších funkcí).

Elementární algoritmické konstrukty

Posloupnost (sekvence):

- krok v algoritmu, který je považován za jednotku (nemusí být elementární operací) a je proveden pouze jednou,
- v případě více posloupností je jasně dáno pořadí jejich zpracování.

Větvení, výběr (selekce):

- podmíněný výběr průběhu algoritmu na základě vyhodnocení výběrové složky.

Cyklus (iterace):

- opakování části algoritmu (těla cyklu), které je prováděno dokud je/není splněna podmínka opakování.

Dílčí algoritmus (podprogram, funkce, metoda):

- část algoritmu, kterou lze použít opakovaně.

Výpočetní problém - příklad

- Úloha: najděte největšího společného dělitele dvou přirozených čísel (víme, co je největší společný dělitel dvou přirozených čísel?)
- Řešení:
 - Popišme postup tak, aby byl použitelný pro dvě libovolná přirozená čísla:
 - označme zadaná čísla x a y a menší z nich d
 - není-li d společným dělitelem x a y , pak d zmenšíme o 1, test opakujeme a skončíme, až d bude společným dělitelem x a y
 - Poznámka:
 - Význam symbolů x , y a d použitých v algoritmu:
 - jsou to proměnné (paměťová místa), ve kterých je uložena nějaká hodnota, která se může v průběhu výpočtu měnit.

Algoritmus pro největšího spol.dělitele

Úloha: najděte největšího společného dělitele dvou přirozených čísel

Přesnější popis:

- Vstup: přirozená čísla x a y
- Výstup: $\text{nsd}(x,y)$
- Postup:
 1. Je-li $x < y$, pak d má hodnotu x , jinak d má hodnotu y
 2. Pokud je d dělitelem x nebo je d dělitelem y , jdi na krok 5
 3. Zmenši d o 1
 4. Jdi na krok 2
 5. Výsledkem je hodnota d
- Sestavili jsme algoritmus pro výpočet největšího společného dělitele dvou přirozených čísel.

Zápis algoritmu NSD v C++

```
#include <iostream>
using namespace std;

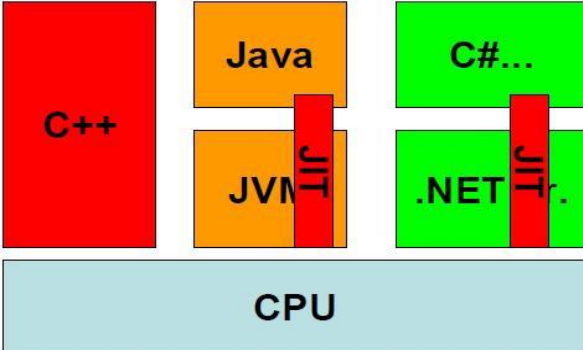
int main() {
    int x,y,d;
    cout << "Zadej dve cisla: "; cin >> x >> y;
    if (x < y) d = x; else d = y;
    while (!(d % x == 0) && (d % y == 0)) {
        d -= 1;
    };
    cout << "NSD(" << x << "," << y << ") = " << d << endl;
    return 0;
};
```

Stručný úvod do OO stylu

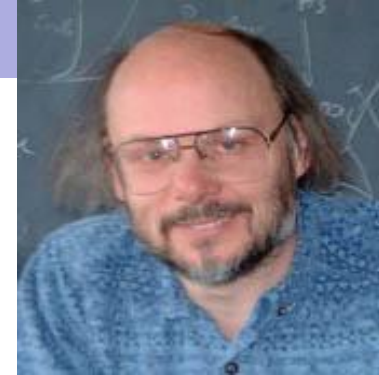
- Objektový model výpočtu si představuje řešení jako sadu objektů, které si posílají zprávy – požadují od jiných objektů vykonání nějaké akce, kterou samy neumí.
- Objekty jsou instance tříd, které shrnují jejich společné vlastnosti a definují jejich funkčnost – metody, které lze nad objekty, příp. s objekty provádět.
- Příklad: Představme si objekt typu fronta zákazníků – je to instance třídy FRONTA. Třída FRONTA umí provést metodu zarad_zákazníka, která do fronty zařadí dalšího zákazníka. Systém, který simuluje obsluhu zákazníků vytvoří nového zákazníka (objekt typu ZAKAZNIK) a nechá si ho zařadit do fronty zákazníků. Z fronty pak vybírá zákazníky, které bude obsluhovat.
- Abychom si mohli předvádět ukázky, řekneme si něco o struktuře objektového řešení a jeho zápisu v C++.

Historie C a C++

- Jazyk C původně vznikl jako systémový jazyk pro operační systém Unix (aby se nemusel psát v assembleru).
- Vycházel z jazyka BCPL (Martin Richards, Cambridge).
- Původní verze: jazyk B (Ken Thompson, PDP-7, 1970).
- Při přechodu na PDP-11: jazyk C (Dennis Ritchie a spol. - 1972)
- 1978 - Kernighan, Ritchie: The C Programming Language (K&R C)
- 1982 - ANSI C - X3J11 (ISO WG14 1990, poslední doplňky ISO/IEC 9899, 1999 tzv. ISO C99).
- Později vznikají různá objektová rozšíření C (Objective C, C++, ..).
- C++ navrhl Bjarne Stroustrup, 1983, AT&T Bell Laboratories.
- 1985 Bjarne Stroustrup: The C++ Programming Language.
- 1990 Bjarne Stroustrup: The Annotated C++ Reference Manual.
- 1998 - ISO WG21, ISO/IEC 14882 (ANSI C++ - X3J16, 1998).
- Současný standard je C++17 (ISO/IEC 14882:2017).



Jazyk C++



- Historie

- 1979 – C s třídami, 1983 – Bjarne Stroustrup, Bell Labs.
- Nadstavba jazyka C (procedurální – program rozdělen do funkcí – princip shora dolů) - zdrojový kód je kompilován do strojového kódu dané platformy (kompilován pro každou platformu/Java – 1 byte code pro všechny platformy + JVM).
- Standardizace 1998 ISO, ANSI: ISO/IEC 14882:2011.
- OOP – z jazyka Simula67 (malé organizační celky = objekty – komunikují přes rozhraní).
- Lze psát i neobjektově.

- Vlastnosti

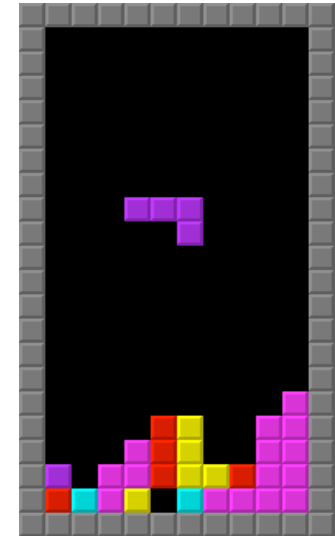
- Vysokoúrovňový - Podporuje objektové programování, šablony, testování, atp.
- Stabilní - Prošel léty vývoje, syntaxe je ustálená, kompatibilita zajištěná.
- Podpora - Od mobilních telefonů po serverové aplikace je neustále nasazován.
- Udržovaný - Jeho vývoj není zmrazený, připravují průběžně nové verze, optimalizují se překladače.
- Podpůrné knihovny - počínaje databázemi, přes mobilní frameworky, grafické knihovny atd.
- Budoucnost - bez ohledu na různé trendy, C++ je rychlý jazyk, který je významným (ne však jediným) jazykem pro operační systémy, grafické aplikace a „embedded“ zařízení.
- Není tak pohodlný, rychlejší jazyky pro vývoj aplikace, nevhodné pro web.

Svět Objektů

- **Objektově Orientované Programování (OOP):**
 - Vychází z intuitivního chápání světa.
 - Základním konceptem jsou objekty.
 - Objekty mají určité *vlastnosti (properties)* a *schopnosti (capabilities)*.
- **Objekty jsou stavební bloky programu:**
 - Program je tedy soubor interagujících objektů.
- **Objekty mohou modelovat **reálné věci**:**
 - auto, hrnek, židle, zeměkoule.
- **Objekty mohou modelovat **koncepty**:**
 - schůzka, shromáždění, telefonát.
- **Objekty mohou modelovat **procesy**:**
 - hledání cesty bludištěm, seřazení balíčku karet.

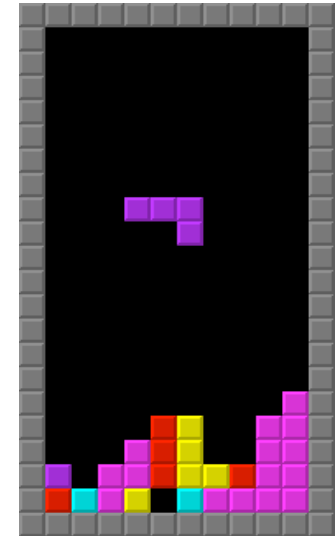
Příklad - Tetris

- Jaké jsou herní objekty?
- Co musí tyto objekty umět vykonat?
- Jaké vlastnosti tyto objekty mají?



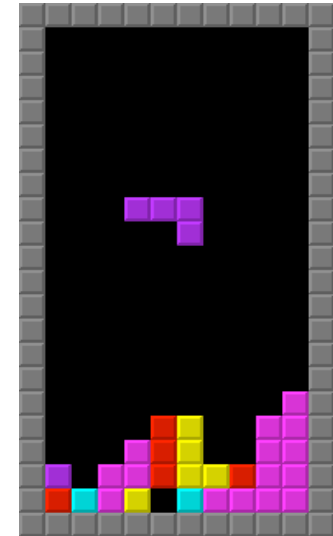
Příklad - Tetris

- Jaké jsou herní objekty?
 - např. dílky a hrací plán
- Co musí tyto objekty umět vykonat?
- Jaké vlastnosti tyto objekty mají?



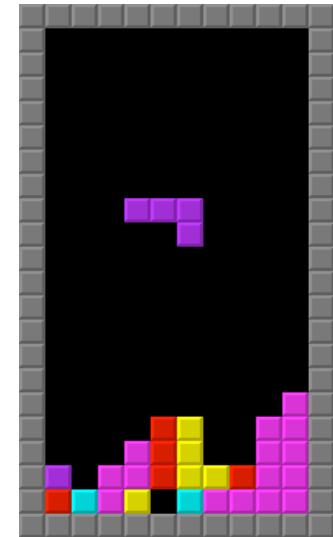
Příklad - Tetris

- Jaké jsou herní objekty?
 - např. dílky a hrací deska.
- Co musí tyto objekty umět vykonat?
 - dílek:
 - být vytvořen, spadnout, zastavit se, otočit se,
 - hrací deska:
 - být vytvořena, odstranit řádek,
 - zkontrolovat konec hry.
- Jaké vlastnosti tyto objekty mají?



Příklad - Tetris

- Jaké jsou herní objekty?
- Co musí tyto objekty umět vykonat?
- Jaké vlastnosti tyto objekty mají?
 - dílek
 - orientace, pozice, tvar, barva,
 - hrací deska
 - velikost, počet řádek.



Objektově orientovaný styl

- Hlavní zásady:
 - Při rozkladu problému specifikujeme nové datové typy (datové abstrakce), tzn. specifikujeme atributy, které budou objekty charakterizovat a dále specifikujeme operace, které se budou s atributy těchto objektů provádět.
 - Pro realizaci datových abstrakcí použijeme objekty a třídy.
- Charakteristika objektu:
 - objekt se může nacházet v různých stavech daných hodnotami svých datových položek (atributů),
 - chování objektu je dáno metodami (operacemi), které jsou pro něj definovány třídou,
 - datové položky a metody objektu jsou dány typem objektu – třídou,
 - datové položky má každý objekt své vlastní,
 - metody jsou společné pro celou třídu a realizují určitý algoritmus.

Schopnosti Objektů

- Schopnosti (*metody*) objektů umožňují provádět specifické akce.
- Metoda objektu musí být vyvolána nějakým objektem (včetně objektu, na kterém je metoda volána).
- Metody objektů dělíme na:
 - konstruktory - inicializují počáteční stav objektu
 - příkazy - mění vlastnosti, tudíž stav objektu
 - dotazy - poskytují odpověď na základě vlastností, stavu objektu
 - destruktory – ruší objekty

Příklad

- Popelnice

- konstruktor: vytvoř popelnici
- příkazy: přidej odpad, vyprázdni se
- dotazy: kolik je odpadu v popelnici?, je otevřeno víko?



Vlastnosti Objektů

- Vlastnosti zajišťují unikátnost objektů:
 - Ovlivňují jakým způsobem se metody na objektu chovají.
 - Některé vlastnosti mohou být konstantní, jiné se mohou měnit.
 - Vlastnosti mohou být také objekty.
- Vlastnosti *objektů* dělíme na:
 - atributy - objekty, které pomáhají *objekt* popsat,
 - komponenty - objekty, které jsou součástí daného *objektu* (objekt se z nich skládá, např. auto *má* kola),
 - asociace - objekty o kterých daný *objekt* ví, ale nejsou jeho součástí (např. auto je v garáži, auto může vědět o garáži).

Stav Objektu

- Stavem *objektu* nazýváme kolekci všech jeho vlastností společně s jejich hodnotami.
- Stav *objektu* se změní pokud se změní aktuální hodnota(y) některé jeho vlastnosti(í).
- Mějme *objekt Auto* s následujícími vlastnostmi:
 - rok výroby,
 - barva.
- Co popisuje stav tohoto *objektu*?
- Může se změnit *barva*? Ano, auto můžeme nechat přebarvit.
- Může se změnit *rok výroby*? Ne, jedná se o konstantní vlastnost.

Třídy a Instance

- Koncept, který jsme zavedli, modeluje každý objekt reálného světa samostatným objektem.
- Jinými slovy, každé auto bychom museli definovat od začátku se všemi vlastnostmi atd.
- Nevýhoda? - Příliš mnoho práce pro programátora objekty specifikovat.
- Velmi nepraktické, objekty mohou mít mnoho společného:
 - Např. auto má víceméně vždy kola, volant, sedadla, barvu, atd...
- Zavedení tříd nám přinese seskupení společných rysů objektů z nich “instancovaných”:
 - společné vlastnosti stačí popsat jednou,
 - poté můžeme vytvářet instance dle libosti, které budou mít tyto vlastnosti společné.

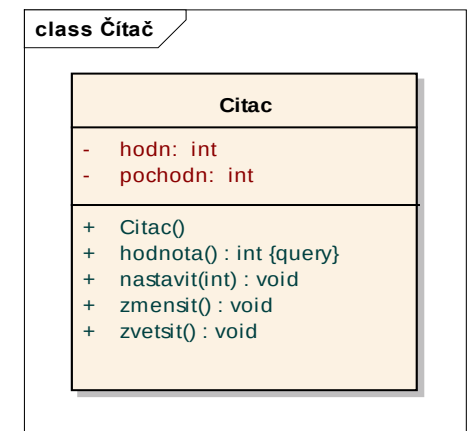
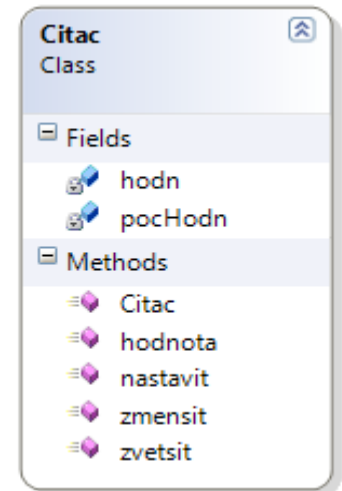
Definice třídy (příklad)

```
class Osoba {  
    int vek; // implicitně private (na rozdíl od struktury)  
public:  
    void nastavVek(int v) { vek = v; };  
    void zobrazVek() { cout << "Vek: " << vek << endl; };  
    void zvysVek() { vek++; };  
};
```

```
Osoba o1;  
//o1.vek = 10; // nelze - věk je privátní položka  
o1.nastavVek(10);  
o1.zobrazVek();  
o1.zvysVek();  
o1.zobrazVek();
```

Základy objektové orientace

- OO analýza a návrh – UML a metodiky.
- OO implementace – OO jazyky.
- Komponentový vývoj – znouvupoužitelnost modelu i kódu.
- Třída = uživatelem definovaný typ (abstrakce), který modeluje část reality.
- Třída = šablona pro vytváření objektů.
- Třída má vlastnosti a chování, nelze k nim přistupovat (měnit je za běhu).
- Objekt = reálný výskyt (instance) třídy, zhmotnění třídy, svět je tvořen objekty.
- Objekt lze vytvořit, používat, měnit jeho vlastnosti i odstranit, má tedy stav.
- Velice zjednodušeně je třída typ a objekt je hodnota tohoto typu.

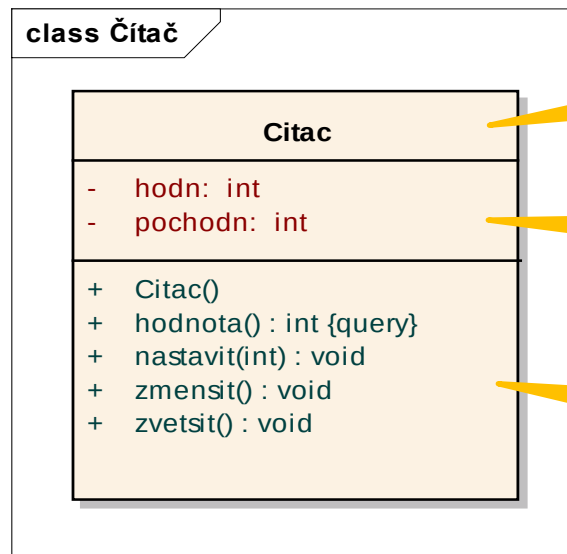


Postup tvorby OO programu

- **Analýza problému**
 - Formulace úlohy.
 - Předpokládané vstupy a výstupy a podmínky pro jejich dosažení.
- **Návrh - rozvaha o algoritmu**
 - Exaktní řešení/přibližné řešení, dekompozice, omezení vstupních dat.
 - Požadavky:
 - paměťová náročnost, rychlost, přesnost.
- **Globální architektura**
 - Určení tříd.
 - Operace prováděné s objekty.
 - Rozhraní pro objekty.
- **Detailní architektura**
 - Návrh tříd: která třída bude řešit kterou problematiku.
 - Datové položky: v jaké podobě chceme data ukládat.
 - Metody: co se bude s daty provádět (objekt minimálně 1 operace), volba formálních argumentů, návratových hodnot a algoritmus.

Příklad: Čítač objektově jako datový typ

- Čítač zavedeme jako datový typ s operacemi:
 - zvetsit, zmensit, nastavit a hodnota
- a s datovými položkami:
 - hodn a pochHodn
- Grafické vyjádření:



**název typu
(třídy)**

**datové položky
(atributy)**

**operace
(metody)**

- Poznámka: hodnota položky pochHodn bude stanovena při vytvoření objektu typu Citac – můžeme vytvářet čítače s různými počátečními hodnotami.

Třída „Citac“

```
/* citac.h */

class Citac {
    int hodn;
    int pocHodn;
public:
    Citac(int ph) { pocHodn = ph; nastavit(); }
    void zvetsit() { hodn++; }
    void zmensit() { hodn--; }
    void nastavit() { hodn = pocHodn; }
    int hodnota() { return hodn; }
};
```

Program s objektem třídy Citac

```
/* citac - objektový styl */
#include „citac.h“
#include <iostream>
using namespace std;

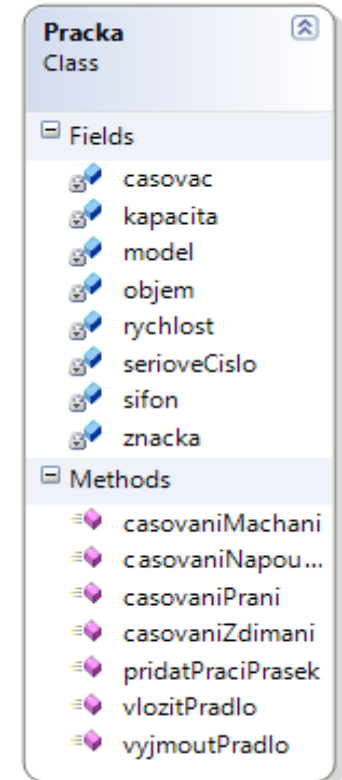
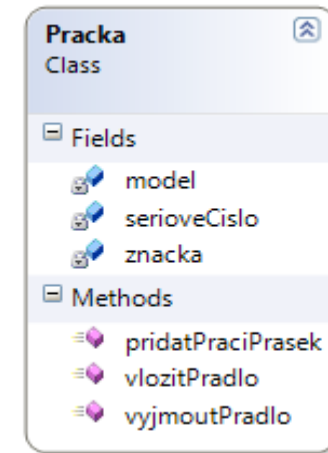
int menu() /* vypise nabidku a precte volbu, pak vrati: 0 ... konec, 1,2,3 ...
operace s citacem */ { ... }

int main() {
    int volba;
    cout << endl << "Citac - objektový styl" << endl;
    Citac citac(0);
    do {
        cout << "Hodnota = " << citac.hodnota() << endl;
        volba = menu();
        switch (volba) {
            case 1: citac.zvetsit(); break;
            case 2: citac.zmensit(); break;
            case 3: citac.nastavit(); break;
        }
    } while (volba > 0);
    cout << "Konec\n";
    return 0;
}
```

Koncepty objektové orientace

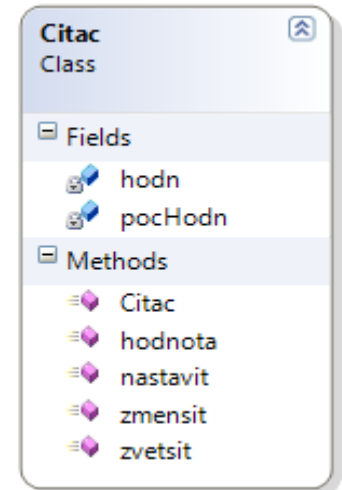
Abstrakce – výběr a zahrnutí jen takových vlastností modelované entity, jež jsou nezbytné pro správnou funkčnost.

- Zanedbání nepodstatných detailů.
- Dle účelu různý pohled na stejnou entitu:
 - entita pračka pro výrobce/entita pračka v prádelně.
- Použití zobecnění - nalezení společného, zastřešujícího pojmu pro skupinu entit:
 - mrkev, pórek, květák -> zelenina.
 - pomeranč, jablko, švestka -> ovoce.
- Třída je nositel abstrakce.



Koncepty objektové orientace (pokr.)

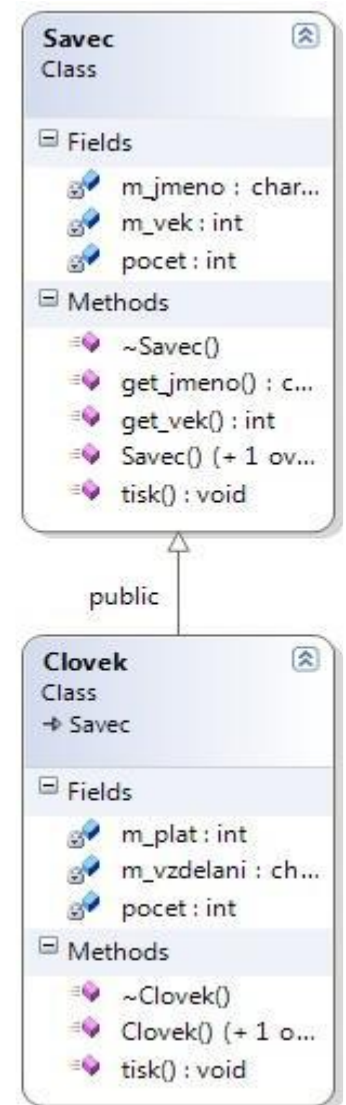
- **Zapouzdření** – rozlišení vnitřní a vnější části třídy:
 - Objekt je černá skříňka.
 - Svazuje dohromady metody a členská data třídy.
 - Data jsou přístupná jen pomocí obalujících funkcí.
 - Zabezpečuje objekt před vnějšími nepovolenými zásahy.
 - Umožní skrýt vnitřní funkčnost objektu před okolím.
 - To jak uvnitř probíhají operace nesdělujeme okolnímu světu.
- **Skrývání informací:**
 - Privátní členská data a metody nelze měnit mimo objekt.
 - Veřejná členská data a metody jsou přístupná i mimo vlastní objekt.
- **Rozhraní - objekt musí nabízet možnost jak s ním komunikovat**
 - Veřejné metody = rozhraní třídy/objektu.
- **Výhoda:**
 - Rozhraní zůstává stejné, mění se jeho implementace podle potřeb.
 - Vnitřek objektů je chráněn.



Koncepty objektové orientace (pokr.)

Dědičnost – nástroj pro dosažení znuvupoužitelnosti:

- Modeluje generalizaci a specializaci.
- Umožňuje odvozovat nové třídy od již existujících.
- Odvozená třída dědí vlastnosti a metody původní třídy, nebo původních tříd.
- Odvozená třída přidává vlastní nové vlastnosti a metody.
- Odvozená třída umí o něco víc než původní třída.
- Odvozená třída může vždy zastoupit původní třídu.
- Původní třída = rodič, базová třída, předek.
- Odvozená třída = potomek.



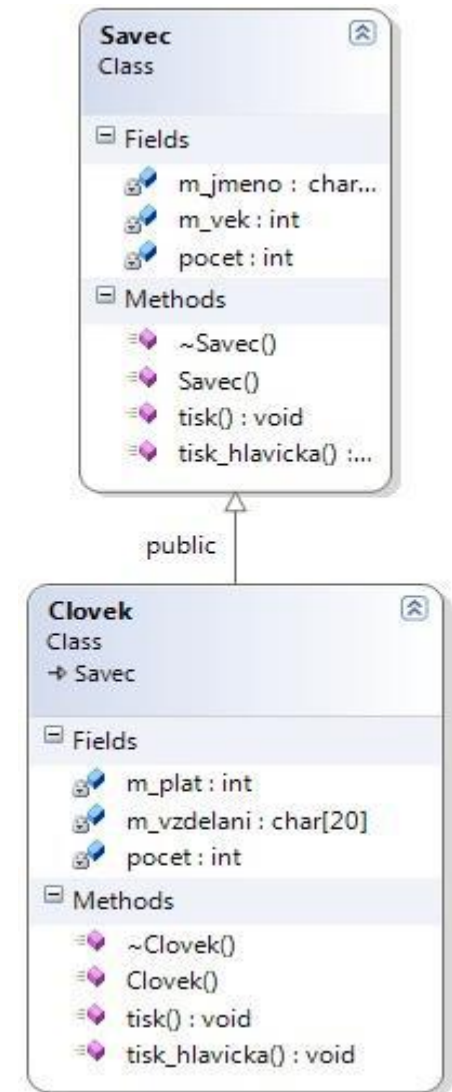
Koncepty objektové orientace (pokr.)

Polymorfismus:

- stejně pojmenované operace (akce, metody) vykonávají různou činnost pro různé entity.

Mnohotvarost:

- Jedno rozhraní – více metod.
- Operace má více implementací
 - volí se na základě entity, která ji provádí (zpracovává).
- Přetěžování metod/operátorů:
 - stejný identifikátor, různý počet/typy argumentů -
`abs()`, `fabs()`, `labs()` => `abs()`
 - `&`, `+`, `atd...`
- Dědičnost rozhraní:
 - rodič definuje rozhraní, potomek implementuje odlišnosti.

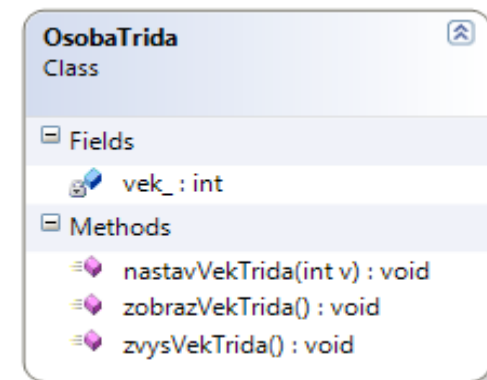


Koncepty objektové orientace (pokr.)

Zasílání zpráv – mechanismus spolupráce/komunikace mezi objekty:

- Objekt zašle jinému objektu zprávu o požadavku na provedení operace.
- Pokud objekt zprávě rozumí, musí ji přijmout.
- Pokud zprávě nerozumí, musí ji odmítnout.
- Po obdržení zprávy tuto operaci oslovený objekt provede.
- Zpráva = volání metody.
- Parametry zpráv = parametry metody.

```
class OsobaTrida {  
    int vek_; //:private  
public:  
    void nastavVekTrida(int v);  
    void zobrazVekTrida();  
    void zvysVekTrida();  
};
```



```
OsobaTrida ot1;  
//ot1.vek_ = 10; // nelze  
ot1.nastavVekTrida(10);
```

Vztahy mezi objekty

Mezi objekty reálného světa existují vztahy - třídy mají **vztahy**

- Pro vztah se používají slovesa (slovesné fráze).
- Jednosměrný, obousměrný (Ovladac zapne TV) (Muz je manžel Zeny, Zena je manželkou Muze).
- Násobnost (Jizdní kolo má 2 kola, Ucitel má více Predmetu) – proměnná/ pole C++.

Závislost (dependency) – argument metody je použitá třída C++

- Nejobecnější (nejvolnější) vztah (typicky objekt A může ovlivnit objekt B).
- Zvire požije Potravu, Zakaznik ovlivní Restauraci.

Asociace – spřátelené funkce a třídy C++/ref. proměnná - argument

- Vztahem (objekt A zná objekt B, A je schopno poslat zprávu B) – obecná slovesa.
- Ovladac zapne TV, Muz je přítelem Muze.

Agregace – třída obsahuje jinou třídu (třídy)

- Silnější typ vztahu – objekt A je složen z objekt(u)ů B(B1..Bn), B existuje i po zrušení A.
- PC je složeno z Klavesnice, Mysi, Monitoru; Auto je složeno z Kol.

Kompozice – třída obsahuje jinou třídu, ale ta bez ní nemůže existovat

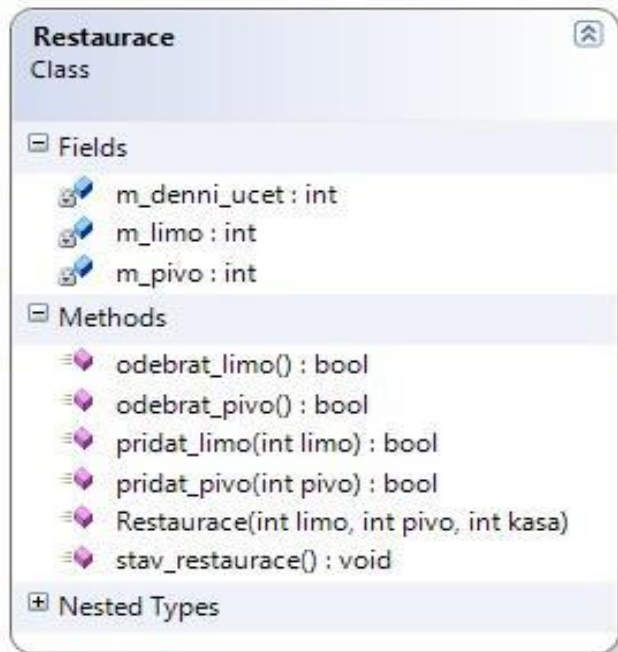
- Silnější agregace – objekt A obsahuje objekt B, existence B je bez A nesmysl.
- Ovladac je složen z Tlacitek, Kosile je složena z Rukavu, Limecku.

Generalizace (Specializace) – veřejná dědičnost C++

- Vztah označovaný slovesem je (is, lépe isA = is a).
- Objekt B je objekt A (je jeho specializací) (je jeho druhem - Clovek a Savec).

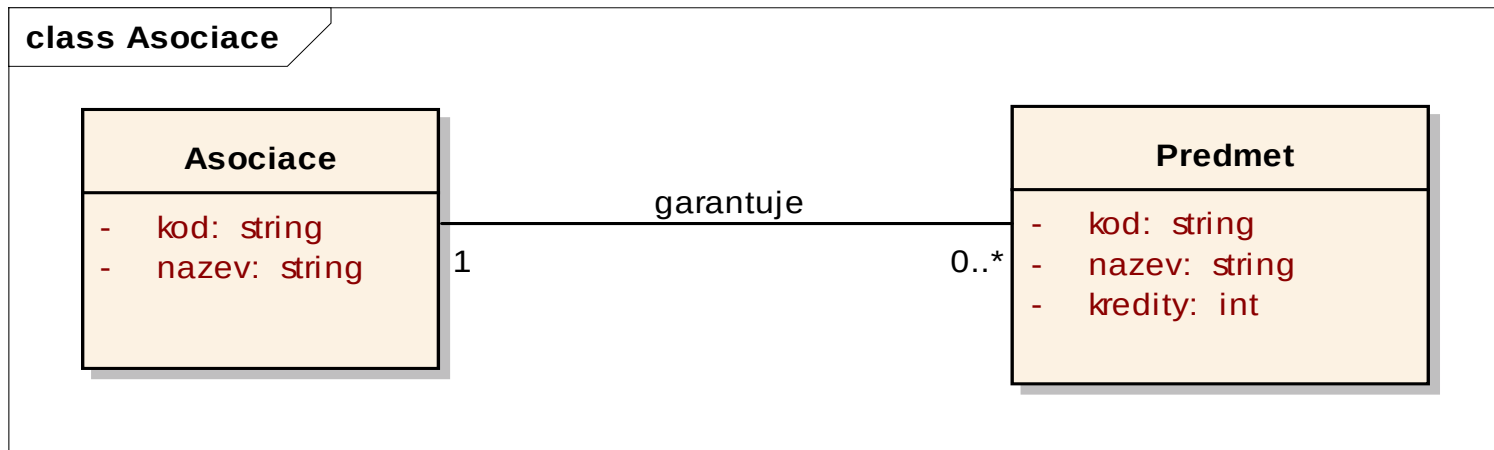
Závislost – C++

- V definici jedné třídy se odkazujeme na třídu jinou:

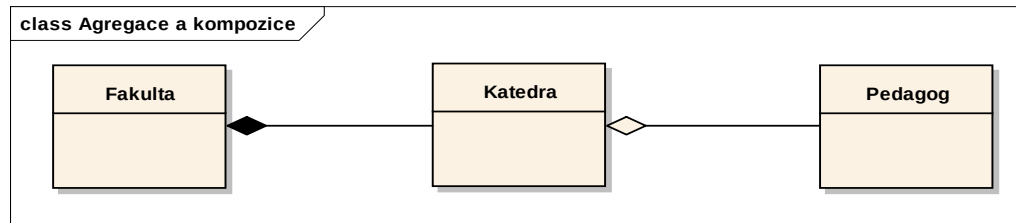
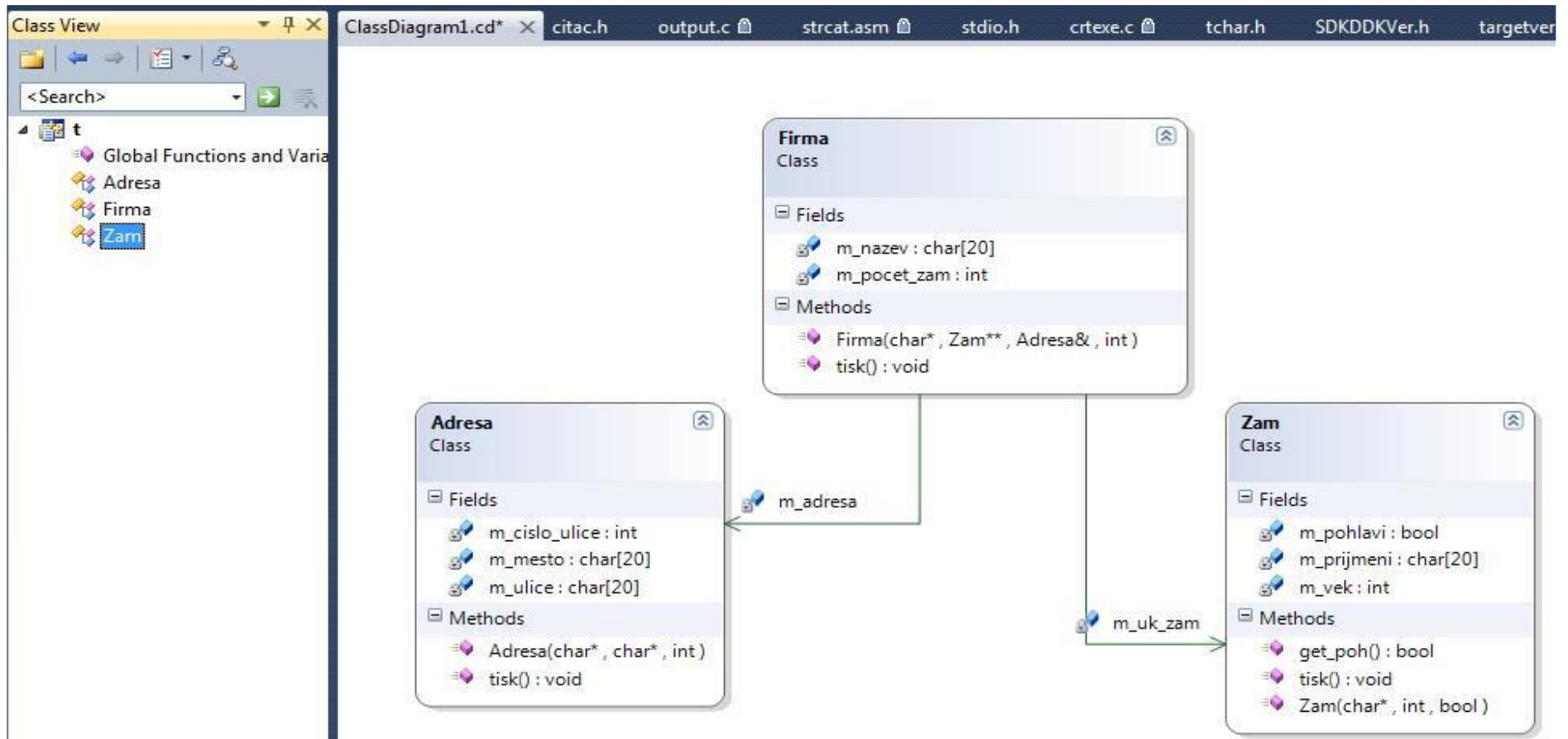


Asociace – C++

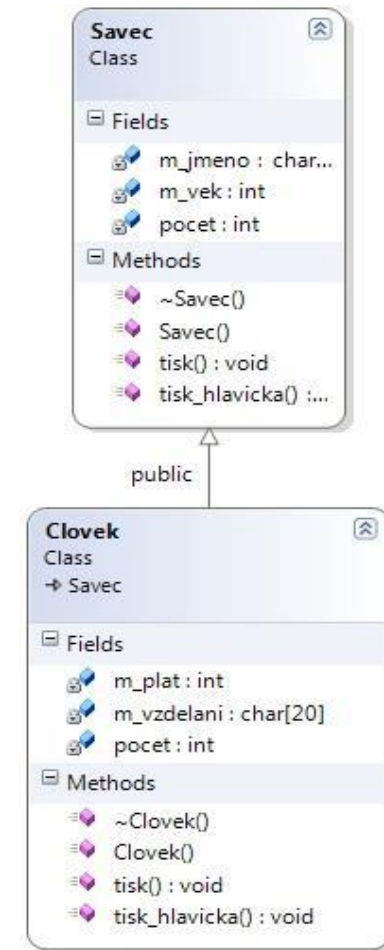
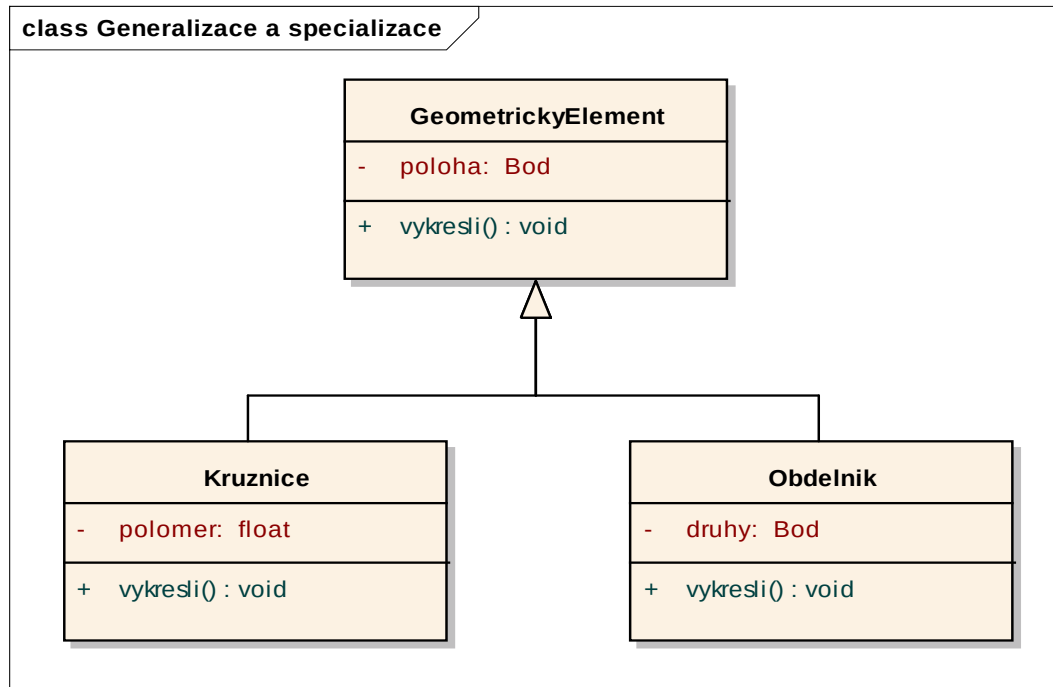
- Násobnost



Agregace (kompozice) - C++

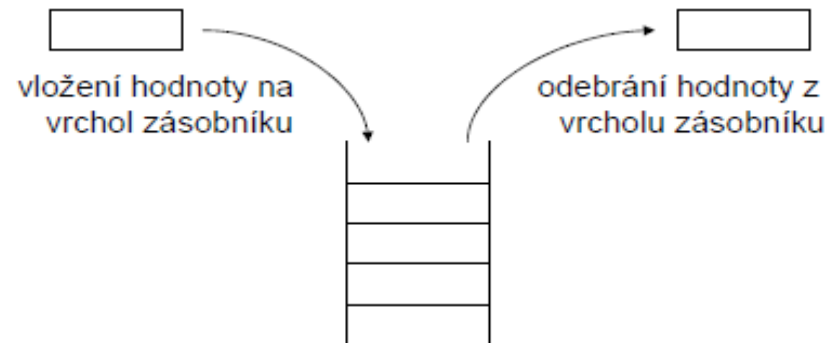


Specializace - C++ (veřejná dědičnost)



Příklad: Zásobník

- Zásobník (stack) je abstraktní datový typ, který umožňuje vkládání a odebírání hodnot, přičemž naposledy vložená hodnota se odebere jako první.
- Zásobník je paměť typu LIFO (zkratka z angl. last-in first-out, poslední dovnitř, první ven).



- Základní operace:
 - vložení hodnoty na vrchol zásobníku
 - odebrání hodnoty z vrcholu zásobníku
 - test na prázdnost zásobníku

Třída realizující zásobník

```
typedef char TypPrvku;
```

```
class Zasobnik {  
public:  
    Zasobnik(int maxDelka = 100);  
    ~Zasobnik();  
    void vloz(TypPrvku p);  
    TypPrvku odeber();  
    bool jePrazdny();  
private:  
    ...  
};
```

- Poznámky
 - v angličtině se operace nad zásobníkem obvykle jmenují push, pop a isEmpty,
 - pro zásobník může být definována ještě operace čtení hodnoty z vrcholu zásobníku bez jejího odebrání (v angl. top).

Příklad použití zásobníku

- Program, který přečte výraz obsahující tři druhy závorek a zkontroluje jejich správné párování:
 - příklad správného párování: { [] () }
 - příklad nesprávného párování: { [(]) }
- Nástin řešení:
 1. Postupně projdeme všechny znaky tvořící výraz. Pro každý znak mohou nastat tři případy:
 - jestliže znak je otevírací závorka, pak jej uložíme do zásobníku,
 - jestliže znak je zavírací závorka, pak ze zásobníku (musí být neprázdný) odebereme naposledy uloženou otevírací závorku a zjistíme, zda odpovídá zavírací závorce,
 - jestliže znak není závorka, pak nás nezajímá.
 2. Po zpracování všech znaků výrazu musí být zásobník prázdný (pokud není, pak chybí zavírací závorky).

Párování závorek

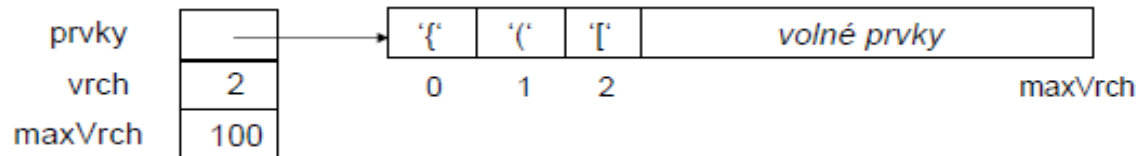
Pomocné funkce:

```
bool jeOteviraci(char z) {
    return z=='(' || z=='[' || z=='{' ;
}
bool jeZaviraci(char z) {
    return z==')' || z==']' || z=='}' ;
}
char zaviraciK(char z) {
    if (z=='(') return ')';
    if (z=='[') return ']';
    if (z=='{') return '}';
    return z;
}
void chyba(char *str) {
    cout << " chyba: " << str << endl;
    system("PAUSE"); exit(1);
}
```

Hlavní program pro párování závorek

```
int main() {
    const int MaxDelka = 100;
    Zasobnik zasobnik(3);
    cout << "zadejte radek se zavorkami" << endl;
    char radek[MaxDelka];
    fgets(radek, MaxDelka, stdin);
    int i, delka = strlen(radek);
    for (i=0; i<delka; i++) {
        char znak = radek[i];
        cout << znak;
        if (jeOteviraci(znak)) zasobnik.vloz(znak);
        else if (jeZaviraci(znak)) {
            if (zasobnik.jePrazdny()) chyba("k teto zavorce chybi oteviraci");
            char ocekavany = zaviraciK(zasobnik.odeber());
            if (znak!=ocekavany) chyba("nespravna zaviraci zavorka");
        }
    }
    if (!zasobnik.jePrazdny()) chyba("chybi zaviraci zavorka(y)");
    cout << endl;
    system("PAUSE");
    return 0;
}
```

Implementace zásobníku pomocí pole



```
class Zasobnik {  
public:  
    Zasobnik(int maxDelka = 100);  
    ~Zasobnik();  
    void vloz(TypPrvku p);  
    TypPrvku odeber();  
    bool jePrazdny();  
private:  
    TypPrvku *prvky;  
    int vrch;  
    int maxVrch;  
};
```

Implementace zásobníku pomocí pole

```
/* zasobnik.cpp */
#include <stdlib.h>
#include <iostream>
using namespace std;
#include "zasobnik.h,,

Zasobnik::Zasobnik(int maxDelka) {
    maxVrch = maxDelka-1;
    prvky = new TypPrvku[maxDelka];
    vrch = -1;
}

Zasobnik::~Zasobnik() {
    delete [] prvky;
    maxVrch = -1;
    prvky = NULL;
    vrch = -1;
}
```

Implementace zásobníku pomocí pole

```
void Zasobnik::vloz(TypPrvku p) {
    if (vrch<maxVrch) prvky[++vrch] = p;
    else {
        cout << "pretečení zásobníku" << endl;
        system("PAUSE");
        exit(1);
    }
}

TypPrvku Zasobnik::odeber() {
    if (vrch>=0) return prvky[vrch--];
    else {
        cout << "odebírání z prázdného zásobníku" << endl;
        system("PAUSE");
        exit(1);
    }
}

bool Zasobnik::jePrázdný() {
    return vrch<0;
}
```

The End